FIDELITYLOOP: CLOSED-LOOP VALIDATION OF ELASTIC LLM-SERVING POLICIES

Anonymous Authors[1]

ABSTRACT

Replay models guide the selection of elastic LLM-serving policies, but accurate request timing and aggregate resource estimates need not preserve deployment decisions. We present FidelityLoop, a closed-loop validation framework that tests policy feasibility and resource use across replay and physical execution. A shared policy interface and identity ledger align requests, lifecycle transitions, and control actions across replay, vLLM, and SGLang, with Vidur as an external service model. Measurements on two A100 GPUs and replay contrasts reveal two failure mechanisms: startup timing can cross a controller's cooldown boundary and alter subsequent actions, while resource errors can cancel in aggregate despite temporal misalignment. Tests with rule-based and replay-trained PPO policies show that a startup proxy falsely rejects 8 of 22 feasible runs in one deployment context and accepts a failing policy in another. These errors change policy selection: retrospective screening selects a candidate that fails all six physical repeats; in another tested pool, it selects a feasible candidate costing 58.95% more than the lowest-cost feasible alternative under the scenario ledger. FidelityLoop links these decisions to execution traces so developers can diagnose why replay and deployment disagree. We release paired traces from 197 physical runs, including 36 preregistered runs, CPU analysis tools, and a fixed-input Vidur replay package under an explicit resource and synthetic-API cost contract.

1 INTRODUCTION

Elastic LLM serving releases idle instances to save resources, but capacity returns only after startup. Queues, running work, and lifecycle transitions then shape subsequent control decisions. A simulator used to evaluate such policies must capture when capacity is available, not just how long individual requests take or how many GPU seconds a run consumes.

FidelityLoop connects prediction, policy decisions, and physical lifecycle execution through a shared contract (Figure 1). It tests whether a replay model preserves the outcomes that matter for policy selection: deadline feasibility and the resources consumed by feasible executions. Our measurements reveal two ways aggregate accuracy can mislead this choice. Startup timing can cross a controller's cooldown boundary and change subsequent actions; resource errors can also cancel in the total while capacity is available at the wrong times. Across policies, arrivals, and serving contexts, controlled physical measurements connect these errors to deployment choices and costs.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

The paper makes three contributions:

1. **A reusable closed-loop validation framework.** FidelityLoop separates policy adapters from lifecycle execution and aligns causal observations, request identities, resource trajectories, and deadline-aware cost ledgers. Its adapters connect rules and learned policies to vLLM and SGLang, and substitute Vidur service predictions in replay. Developers can change policies or backends while retaining comparable evaluation records.

2. **Mechanisms of replay-to-deployment disagreement.** Held-out physical runs distinguish service, aggregate-resource, and trajectory fidelity. Event-level analysis and controlled replay contrasts expose startup–cooldown interactions and cancellation in aggregate resource error, explaining why local accuracy need not preserve closed-loop behavior.

3. **Decision consequences and reproducible evidence.** Retrospective policy selection connects false accepts and false rejects to failed deployments and higher costs within tested candidate pools. Rule-based and frozen PPO policies, additional arrivals, serving-stack changes, and a model-size port exercise the validation framework across contexts. A guard case study quantifies deadline protection and resource–latency trade-offs. We release

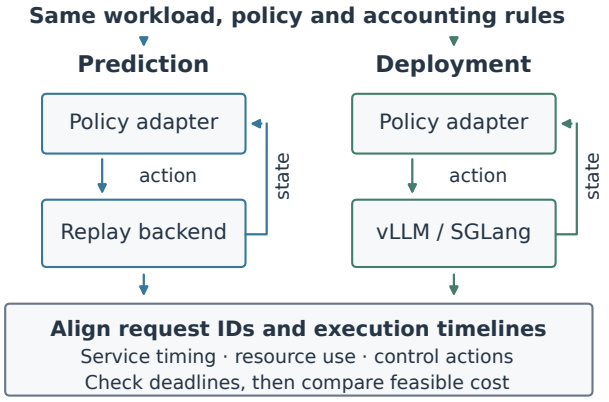


Figure 1. FidelityLoop runs the same policy contract in separate replay and physical control loops. Each adapter reads its own backend's causal state. Aligned records expose errors in both predictions and policy choices; physical outcomes are used for validation after execution.

paired traces and CPU tools for inspecting these outcomes and replaying the fixed Vidur-backed cases.

The release provides CPU interface checks, a fixed-event cost verifier, and a paired trace corpus[1]; the supplement documents their reproduction scopes.

2 Background and Motivation

We consider mixed online and offline LLM workloads served by a limited GPU pool, where elastic scheduling aims to reduce resource use while meeting request deadlines. Evaluating every candidate policy on physical hardware is costly, motivating a framework for replay-based policy development and training followed by physical validation. FidelityLoop connects these stages through a shared policy interface and consistent lifecycle and deadline accounting, so that savings predicted in replay can be checked against deployment outcomes.

Preliminary measurements reveal the problem: calibration reduced prediction error for matched local requests while increasing error in occupied GPU time under dynamic scheduling. These observations precede and motivate the preregistered held-out study.

The lifecycle diagnosis separated the old 69.219 s startup proxy from the earlier window launch-to-ready (25.19 s) and setup (46.80 s) contexts. When the proxy crossed the 60 s cooldown boundary it changed a target decision, motivating the locked sequence of calibration, lifecycle checks, and held-out feasibility tests.

[1] https://anonymous.4open.science/r/FidelityLoop-144B/

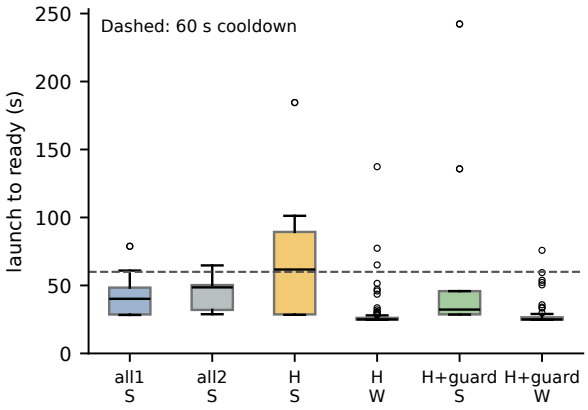


Figure 2. Observed startup times in the held-out runs. Labels use S for setup and W for window scale-out. The dashed line marks the 60-second controller cooldown; boxes summarize observed launch-to-ready times. Release and cleanup are separate ledger events. Collapsing these contexts into one endpoint can alter closed-loop actions.

Figure 2 summarizes this lifecycle boundary; the held-out selection procedure is fixed separately in Section 5.

3 Related Work

Vidur and Charon model LLM execution to guide configuration search; Clockwork targets predictable request timing (Agrawal et al., 2024a; Yang et al., 2026; Gujarati et al., 2020). FidelityLoop's evaluation unit is an elastic policy rollout: its deadline classification, resource and control trajectories, and resulting deployment choice. Its shared policy interface and identity ledger align these outcomes between replay and physical execution.

Cold-start characterization decomposes initialization latency, while ServerlessLLM reduces startup overhead (Kabakibo et al., 2026; Fu et al., 2024). We study how errors in that latency change subsequent capacity decisions and deadline outcomes; the replay endpoint contrast holds the engine and loading path fixed. vLLM, SGLang, and Sarathi-Serve optimize request execution (Kwon et al., 2023; Zheng et al., 2024; Agrawal et al., 2024b); we validate elastic policies across execution backends and replay service models.

Provisioning and scheduling work connects workload estimates to SLOs (Crankshaw et al., 2020; Zhang et al., 2019; 2023). Our comparisons evaluate replay models as policy-screening tools: whether their feasibility and resource predictions support deployment choices. The guard and frozen PPO checkpoints provide rule-based and learned-policy cases, respectively.

Policy design provides a complementary layer. Llumnix dynamically reschedules LLM requests (Sun et al.,

2024); AAPA predicts scaling needs for serverless workloads (Zhang et al., 2025b); KIS-S learns GPU-inference autoscaling with PPO (Zhang et al., 2025a). PPO, the standard clipped policy-gradient method (Schulman et al., 2017), provides a learned-policy case for our framework. We train policies in replay and freeze their checkpoints to test whether replay-selected policies remain feasible in physical execution.

4 Problem and Metrics

The system has two accelerators, online arrivals with a 60 s deadline, and 120 offline jobs per 2,100 s observation window with a 300 s deadline. Online requests run locally or use a delay-only synthetic API sink; offline jobs run locally. Arrivals span 1,800 s; execution and accounting continue for another 300 s to cover the remaining deadline horizon. The primary window cost, in scenario USD, is

$$C = .001G + .002S + .001D + 10^{-6}I + 3\times10^{-6}B + .01M, \quad (1)$$

where G is occupied GPU seconds, S, D are starts and shutdowns, I, B are synthetic API input and output budgets, and M counts offline deadline misses, including unfinished requests whose deadlines have passed. Setup and cleanup are reported separately and are not folded into the primary model gate.

Completed means a recorded completion by the observation cutoff; it may be *on time* (finish time $\leq$ deadline) or *late* (finish time $>$ deadline). An unfinished request has no completed terminal by the cutoff. Unfinished or censored work is retained in the arrival denominator. The capacity gate G_F requires all2 (two GPUs throughout) to complete every online and offline request on time. P1 requires 120/120 offline requests on time, at least 99% online requests on time, and no unfinished request; P1+ additionally requires every online request on time. P2 is a ratio-of-means saving against all1 (one GPU throughout) computed within each window and then averaged equally across the three windows.

5 Workload Construction

Online arrivals in the main study come from the successful-request subset of BurstGPT, a public trace of production LLM requests (Wang et al., 2024). We read all 1,404,294 source rows and retain source hashes and row identities. The trace has no session identifier, so the split separates arrival times rather than sessions. Locally routed requests execute actual GPU inference with fixed-corpus prompts and registered token budgets. The experiment reproduces arrival-time patterns, not the original users' prompts or semantic tasks.

The three held-out windows are selected once from the forward pool after the old windows plus a 30-minute protection band. A 60 s morphology filter defines the steady, recovery, and burst-offline classes (even, declining, and bursty online arrivals); a fixed SHA ordering takes the first eligible candidate in steady–recovery–burst order, subject to the registered separation constraint. Online rows are thinned with the inherited integer threshold, while the 120 offline jobs are constructed at fixed 15 s spacing and 256-input/256-output token budgets. The two salts and all thresholds are fixed in `s4_generation_lock.json`; generated inputs are locked before replay and never replaced to improve a result.

Table 1. Selected window arrival shape at 60 s bins. Each cell is raw/derived; CV and peak-to-mean use the full 1,800 s window, including empty bins.

Window	arrivals	CV	peak/mean	lag-1
steady	5461 / 1444	0.378 / 0.412	1.566 / 1.641	0.772 / 0.797
recovery	1141 / 334	1.444 / 1.452	3.550 / 3.862	0.906 / 0.892
burst-offline	663 / 177	1.951 / 1.935	6.652 / 6.780	0.810 / 0.754

Table 1 reports the arrival shape at the registered 60 s scale. The 1 s diagnostics are retained in the manifest but do not define the class. The values describe the selected derived windows, not the full source population.

The policy deployment study uses two separately generated and locked windows from the same construction pipeline, with no row overlap with the three held-out windows; the campaigns remain distinct and are never pooled. The screening, engine, and 14B studies reuse these two windows. The recovery window contains 358 derived online arrivals and 120 fixed offline arrivals (478 total); the steady window contains 944 online and 120 offline arrivals (1,064 total). The deployment lock binds window identities and input hashes; online timestamps and the offline construction above remain fixed.

A separate external-validity study uses Azure conversation arrival traces associated with Splitwise (Patel et al., 2024); its workload construction and load-boundary results are reported in the supplement.

6 Experimental Setup

The preregistered held-out study used two NVIDIA A100 80GB PCIe GPUs at a university computing facility, with one independent tensor-parallelism-1 instance per GPU and Qwen2.5-7B-Instruct. The serving stack was vLLM 0.9.0 (Kwon et al., 2023) with PyTorch 2.7.0+cu126, Transformers 4.51.1, tokenizers 0.21.1, and NumPy 1.26.4. Instances used FP16, maximum model length 4096, maximum batched tokens 16,384, maximum sequences 1,024, four local request slots, chunked prefill enabled, prefix caching disabled, eager execution enabled, and GPU memory uti-

lization 0.8. Readiness required the per-process health probe and the dual-ready barrier; cleanup required verified process release on both devices. These values are bound by each run's environment snapshot and GPU identity record.

The deployment-adapted model (Local) was calibrated and frozen before held-out execution. The historical Legacy-refit–Local comparison spans different hardware environments (A800 and A100), so it does not isolate the effect of calibration.

The deployment study locks a separate runtime with the same Qwen/vLLM stack: one all2 anchor and five frozen PPO arms per window, with three repeats per PPO arm (32 runs). Checkpoint and tensor hashes bind policy identity. An earlier 38-run mixed-policy study covers rules, guards, and PPO under another runtime. The screening study evaluates six previously undeployed checkpoints on the same two windows, each on a separate dual-A100 allocation: one all2 anchor and two repeats per arm (26 runs). These runtime groups remain separate even where checkpoints are shared.

The engine study transfers six policy arms to SGLang 0.4.8, PyTorch 2.7.1, and Transformers 4.52.3 on two A100 80GB PCIe GPUs. It retains the Qwen weights, windows, policies, and accounting contract: two repeats per arm/window plus two all2 anchors (26 runs). The supplement documents the full-stack change and token-input compatibility.

The arrival study retains the SGLang/7B stack and frozen predictors on three additional windows (27 cells, including 24 policy cells). The 14B study ports the contract to Qwen2.5-14B-Instruct/vLLM on the two locked windows (12 cells), freezing two predictors that differ only in startup; neither extension retrains PPO or searches policy parameters.

7 METHOD

Predictive replay and physical execution share a policy contract. Adapters read observable queues and device states and return capacity targets; the guard may also select dispatch priority. The backend handles readiness, draining, and verified release, while request identities align predictions, decisions, lifecycle states, resources, and costs. A separate CPU verifier recomputes feasibility and cost from raw records. Engine adapters translate launch, readiness, inference, and release operations into this contract, allowing a serving backend to change without redefining policy observations or evaluation metrics. By default, ready local slots serve online requests before offline jobs. Online requests still waiting 10 s after arrival can enter the synthetic API sink, which admits up to eight concurrent requests and completes each on a 5 s timer. The guard can prioritize at-risk offline jobs; PPO changes capacity while retaining the default dispatch and API rules.

Table 2. Frozen replay parameters in seconds, rounded to three decimals. Listed models share a 69.363 s setup proxy outside the primary window ledger.

Model	Service	Startup	Shutdown
Share	equal-share	30.000	7.059
Lookup-69	measured lookup	69.219	7.059
Lookup-30	same lookup	30.000	7.059
Local	same lookup	33.497	2.224
Legacy-refit	same lookup	25.196	2.325

The policy and backend adapters have separate interfaces; the supplement reports their implementation footprint. CPU checks preserve historical request/event ledgers, and causal replay reproduces all 29,580 logged decisions from 16 same-runtime runs. A Vidur service backend (Agrawal et al., 2024a) supplies native scheduling and token execution while FidelityLoop retains admission, lifecycle transitions, and the frozen guard. A Qwen2.5-7B/A100 service screen precedes the retrospective policy comparison in Section 8.1; the supplement provides profiles and parameters.

7.1 Replay models

The replay models share an executor but differ in service and lifecycle timing (Table 2). Share uses an equal-share service model and an assumed 30 s startup. Lookup-69 uses a measured service lookup and a historical 69.219 s startup proxy; Lookup-30 restores the 30 s assumption. Local combines the lookup with deployment-adapted lifecycle parameters; Legacy-refit retains a bounded refit from the earlier environment. Share–Lookup-30 isolates the service model, whereas Lookup-69–Lookup-30 isolates startup; Share–Lookup-69 changes both.

Share assumes 1024 input and 128 output tokens/s, shared equally among concurrent requests. The calibrated lookup uses median measured wall times per input/output/concurrency cell, interpolating in input length and concurrency (one to four requests per instance). These four lookup-based models share the values. Supplement Section B.1 details measurement sources, interpolation, and lifecycle fitting. Model bytes, tokenizer, workload identities, and the candidate grid are locked before held-out execution.

For the engine study, Lookup-SG replaces Lookup-69's startup proxy with the median of 20 SGLang scale-outs measured while the other GPU serves a fixed background load: 27.568 s. It retains Lookup-69's service lookup and shutdown parameter. Lookup-69, Lookup-30, and Lookup-SG predictions are frozen before the policy runs; the guard retains its separate safety estimates.

The arrival study reuses these models unchanged. The 14B study freezes 14B-Hist and 14B-Local with freshly mea-

sured service, setup, shutdown, and guard parameters; only startup differs (69.219 versus 46.067 s).

For each model prediction and physical repeat of a window, the dimensionless lifecycle loss is

$$L = \frac{0.5|\Delta G_{\text{occupied}}| + 0.25|\Delta G_{\text{active}}|}{2T} + \frac{0.25|\Delta Q|}{TN}, \quad (2)$$

Here G_{occupied} sums per-GPU time from launch issuance to verified release, while G_{active} sums time in the active state, including idle time; both are measured in GPU seconds. The difference Δ is prediction minus observation, $T = 2100$ s for formal windows, and N counts *all* online and offline arrivals. Q is the time integral of the released-but-undispatched queue, in request seconds. Unfinished requests remain in N and contribute queue time while queued up to cutoff. Two devices give the $2T$ normalization. We first average losses over repeats within each window, then weight windows equally; we do not pool requests or restrict N to matched-local support. L measures prediction error, separately from scenario cost C; the shorter development horizon is stated with that experiment.

7.2 PPO policy contract

The learned policy is trained only in the CPU replay environment, with Lookup-69 or Lookup-30 as the training environment; the physical serving loop is used only after a checkpoint is frozen. At each one-second decision boundary, its 30-dimensional causal observation contains normalized time and target state, queue counts and ages/slack, device lifecycle states and ages, running-job summaries, and the API queue. The capacity action proposes a target of 0, 1, or 2 GPUs; the registered 60 s cooldown filters that proposal before execution, while the offline action head is fixed to its sole legal value. The reward is the negative incremental scenario-ledger cost, so training trades occupied resources, lifecycle events, API use, and deadline misses under the declared contract.

Training uses the three development arrival regimes and paired seeds from the locked CPU PPO contract. The five deployment checkpoints are selected mechanically with Lookup-30 replay, guard disabled, P1+ required in every validation window, and sorting by equal-window mean cost, checkpoint index, then seed. PPO-1 and PPO-2 were trained with Lookup-69; PPO-3–PPO-5 with Lookup-30. Checkpoint names denote fixed identities and are reused across studies. Training settings, data partitions, and checkpoint coordinates are detailed in the supplement's Frozen PPO training and selection contract section.

7.3 Controller and guard

H uses up-threshold 8, down-threshold 0, and a 60 s cooldown. At each one-second tick, H reads queue and running counts plus its target/cooldown state, not the predictor's startup parameter. The guard's lifecycle-delay estimates use separately frozen safety parameters, distinct from the Lookup-69/Local simulator endpoints. The guard observes only released queues, device states, and running-job ages; future arrivals and true completion or readiness times are unavailable. Let P_t be the pending offline request IDs and b_t H's current target after its decision. The projection $F_j(k, r)$ estimates request j's finish with k devices and reservation setting r, using frozen lifecycle delays, running-job residuals, and concurrency-four service estimates. It schedules pending jobs by deadline then request ID over four slots per device. Online backlog precedes offline work unless reservation ($r = 1$) assigns one slot to offline jobs; without online backlog, offline work can use all slots. The supplement's "Guard implementation map" specifies device selection and overdue-estimate fallbacks. These estimates are not proved completion-time upper bounds.

For the full guard ($r = 1$), margin $m = 5$ s and previous risk-ID set A_{t-1}, the update is

$$\begin{aligned}
R_t &= \{j \in P_t : F_j(\max(1, b_t), 1) + m + 1 \geq d_j\}, \\
A_t &= (A_{t-1} \cap P_t) \cup R_t, \\
K_t &= \{k \in \{1, 2\} : \forall j \in P_t,\ F_j(k, 1) + m \leq d_j\}, \\
u_t &= \begin{cases} b_t, & A_t = \emptyset, \\ \max(b_t, \min K_t), & |A_t| > 0,\ |K_t| > 0, \\ 2, & |A_t| > 0,\ K_t = \emptyset. \end{cases} \quad (3)
\end{aligned}$$

Here d_j is the deadline. Triggering includes a one-second lookahead and uses $\geq$; candidate feasibility uses $\leq$ without that extra second. Empty K_t with active risk sets an `infeasible` flag. A latched ID is removed only when it leaves the pending queue, not when a later estimate becomes safe. The target never falls below *this tick's* H target. A guard raise can bypass H's cooldown and writes u_t and the current time back to H's target and last-change state; it does not guarantee global deadline feasibility.

While A_t is nonempty and both queues have work, dispatch reserves at most one offline slot. An already-running offline job leaves other slots online-first. Otherwise, for head-request slacks $s_x = d_x - t - \widehat{p}_x$, online wins only if $s_{\text{on}} \leq m$ and $s_{\text{on}} \leq s_{\text{off}}$. Without online backlog, offline dispatch is unrestricted. No running request is preempted; later scale-out must wait for readiness.

The registered development search covered 26 controller configurations, including guard variants. Selection preceded held-out input access and returned the preregistered H+guard configuration; its duplicate arm was merged into the four-arm formal matrix to avoid pseudo-replication.

7.4 Evidence protocol

The held-out study evaluates all1, all2, H, and H+guard on steady, recovery, and burst-offline windows, with three repeats per arm (36 formal runs). CPU verification uses the original acceptance code and lock-bound sources. Two operator-aborted attempts remain in history but not formal statistics. Metrics use run and window units. Raw-event recomputation checks terminal counts, lifecycle occupation, and costs against sealed summaries. A technically valid negative result remains in the matrix.

The fixed gate order starts with full-capacity feasibility (G_F), then tests occupied-resource error, deployment-adapted lifecycle loss, and matched-request service fidelity. Target agreement is diagnostic. P1/P1+ and P2 then assess policy feasibility and cost, retaining H's failures as control evidence. This order separates model validity from the appeal of a policy's predicted savings.

8 RESULTS

We first establish the prediction-level mismatch that motivates the framework. We then test replay feasibility in policy deployments, across new checkpoints, and across serving contexts. The guard case study connects these validation results to the resource–latency trade-off.

8.1 Cross-level prediction fidelity

On fixed H, Local's occupied MAE is 67.94 GPU seconds versus 543.51 for Share (87.50% lower). Lookup-30's improvement over Lookup-69 implicates startup. Legacy-refit reduces equal-window L by 34.24% relative to Local, but Local uses deployment-adapted endpoints while Legacy-refit retains the earlier environment snapshot. This historical comparison does not isolate the effect of fitting.

In fixed all1/all2 tests, maximum service MAE is 0.064 s for all four lookup-based models, versus 3.016 s for Share. On dynamic arms, the minimum common support across model/route projections is 122 requests and the maximum common-support MAE is 0.129 s for Lookup-69, 0.112 s for Lookup-30, 0.110 s for Local, and 0.112 s for Legacy-refit; Share reaches 4.510 s. Common support is the intersection of request IDs completed locally in the compared predictions and physical runs; coverage and routing discrepancies are reported in the artifact.

Table 3 reports window-wise occupied-resource errors for fixed-H replay, distinct from the route-conditioned matched-ID checks.

The Lookup-69-versus-Share comparison reproduces the exploratory divergence: maximum matched-ID service MAE falls from 4.510 to 0.129 s (about 35-fold), while equal-window occupied-resource MAE rises from 543.51 to 891.98 GPU seconds (64.1%). This deterioration is not uniform across windows. Holding service fixed, Lookup-30 lowers occupied MAE to 76.19 GPU seconds, isolating startup.

Table 3. Occupied-resource MAE (GPU seconds) by window and model. L is the equal-window lifecycle loss defined above.

Model	steady	recovery	burst	Occupied MAE	L
Share	1151.97	244.08	234.48	543.51	0.09403
Lookup-69	964.14	864.20	847.60	891.98	0.12162
Lookup-30	67.18	78.30	83.08	76.19	0.01353
Local	66.34	66.95	70.55	67.94	0.01353
Legacy-refit	43.51	65.74	69.34	59.53	0.00890

Table 4. Steady-window fidelity: means over the same three physical repeats. Resource errors are GPU seconds; target disagreement is a percentage of decision ticks.

Model	Total error	Trajectory error	Target diff. (%)
Lookup-30	67.18	114.32	1.03
Local	66.34	1390.50	66.73

Retrospective analysis of the frozen trajectories explains this endpoint effect and its limits (Supplement Section M). In all three windows, Lookup-69/Lookup-30 targets first diverge 60 s after a common instance start: Lookup-69 remains unready with higher queue/running pressure; Lookup-30 is ready. This simulator contrast is not a physical intervention on startup or cooldown. Six burst/recovery physical repeats first disagree in target with predictions at 3 s, before any window restart. Conversely, steady repeats 1 and 3 match Lookup-30 targets throughout despite state, routing, and occupation errors.

Aggregate agreement can mask temporal misalignment. In steady, Local and Lookup-30 have nearly equal total occupation MAE, but Local's mean time-integrated absolute error in occupied GPU count is 1390.504 versus 114.324 GPU seconds (Table 4). Positive and negative count errors cancel in the total; their timing still matters to control. Together with the event-level counterexamples, this shows why neither startup correction nor target agreement alone establishes fidelity. The supplement retains all 36 prediction/physical pairs and 12 model/window means; this diagnosis neither changes the resource-error criterion nor reselects a model. The mixed-policy results come from a separate runtime.

Replacing the service lookup with Vidur's profiled model tests the same validation contract with an external predictor. Twelve retrospective CPU rollouts retain Local's lifecycle parameters and the original guard, and reuse all 36 physical

Table 5. Policy deployment on two A100 GPUs. Costs are mean scenario USD; R/S report P1+ passes/runs in recovery/steady. The last column is cost reduction relative to all2; † marks infeasible savings.

Arm	R	S	Cost R	Cost S	Cost ↓ (%)
all2	1/1	1/1	4.200	4.200	0
PPO-1	3/3	3/3	2.357	2.449	42.8
PPO-3	3/3	3/3	2.073	2.084	50.5
PPO-4	3/3	3/3	2.051	2.103	50.5
PPO-5	0/3	0/3	1.899	2.090	52.5†
PPO-2	0/3	0/3	2.730	2.950	32.4†

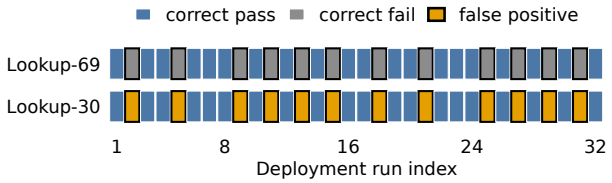


Figure 3. Replay-to-deployment feasibility for all 32 deployment cells. Blue and gray tiles are correct pass/fail directions; orange tiles are Lookup-30 false positives. Black outlines mark cells that physically fail P1+.

runs. They reproduce H's recovery misses (116/120 offline jobs on time versus 113–116 physically) and the guard's 120/120. For H and H+guard in steady, median occupied-resource absolute percentage error (APE) is 2.47%, yet target agreement is 33.0–35.5%; guard recovery has 14.74% occupied-resource APE. Thus the external backend preserves the key deadline contrast while revealing residual timing errors (Supplement Section A.3).

These rule-policy tests establish the need to check both outcomes and trajectories. We next let frozen learned checkpoints drive the same physical lifecycle and compare their predicted and observed feasibility and cost.

8.2 Policy deployment and screening

The deployment study tests whether checkpoints selected in replay remain feasible in physical execution. In its re-provisioned runtime, 196 of 218 window restarts that reached readiness across the 30 PPO runs took over 60 s. All 32 cells are technically valid; four earlier technical failures remain in the audit trail. PPO-4 is the preregistered reference checkpoint.

Three PPO arms pass all six physical repeats: PPO-1, PPO-3, and PPO-4; both all2 anchors also pass. The three feasible PPO arms have scenario costs 42.8–50.5% below all2, but this comparison is conditional on the declared ledger and synthetic API sink. PPO-2 is a decisive negative case: it averages only 4–7 on-time offline requests out of 120 and leaves about 80 unfinished, despite lower cost. PPO-5 is a sharper boundary: it completes the offline stream eventually, but roughly 11 of 120 offline requests miss their deadline per repeat, so its lower cost is not feasible saving. Online requests are on time in every cell; deployment failures are therefore isolated to the offline deadline path rather than hidden by an online regression.

Table 6. Replay-to-deployment agreement and cost MAE (scenario USD). Mixed-policy and deployment include anchors and failures; screening includes its 24 policy runs.

Study	Replay	Agreement	Cost MAE
Mixed-policy	Lookup-69	36/38	0.0969
Mixed-policy	Lookup-30	21/38	0.6998
Deployment	Lookup-69	32/32	0.0804
Deployment	Lookup-30	20/32	0.4657
Screening	Lookup-69	18/24	0.3112
Screening	Lookup-30	24/24	0.0163

Table 6 compares each run with its frozen replay: Lookup-69 correctly predicts all 20 passes and 12 failures, whereas Lookup-30 predicts all cells feasible and misses both negative checkpoints. For each replay model, these 32 comparisons reuse 12 policy/window predictions; the physical repeats are not independent predictions. The two replay models have window-cost MAE of 0.0804 and 0.4657 scenario USD, respectively. PPO-2 exposes a startup–cooldown interaction: the 60 s cooldown begins at the capacity-target change. In Lookup-30 replay, readiness precedes cooldown expiry, leaving time to serve queued work; in Lookup-69 replay and most physical restarts, readiness follows expiry, allowing almost immediate scale-down. Across the six physical PPO-2 runs, 72 of 73 restarts taking over 60 s enter draining within two seconds of readiness (Supplement Section G.2).

The earlier 38-cell mixed-policy study contains rule, guard, and PPO arms on a separate runtime. Lookup-69 also has higher feasibility agreement and lower cost MAE there (Table 6); the campaigns are not pooled. This repeated agreement motivates testing Lookup-69 as a screening rule on new checkpoints.

Both training environments yield passing and failing deployment checkpoints. The screening study tests Lookup-69 on six previously undeployed checkpoints in a new execution context. From 90 eligible candidates, Lookup-69 predicts 67 feasible on both known windows and rejects 23. A fixed ordering selects three distinct trajectories from each group; Lookup-30 does not enter selection. All six pass every physical run (two windows, two repeats), as do both all2 anchors. Lookup-69 therefore falsely rejects three feasible candidates,

Table 7. Engine study: frozen predictions versus 24 SGLang policy runs. False accepts are out of two infeasible runs; false rejects out of 22 feasible runs. Cost MAE includes the declared deadline penalty, in scenario USD.

Replay	Agreement	False accepts	False rejects	Cost MAE
Lookup-69	16/24	0/2	8/22	0.5420
Lookup-30	24/24	0/2	0/22	0.0468
Lookup-SG	24/24	0/2	0/22	0.0427

while Lookup-30 accepts all six. Across the 24 policy cells, Lookup-30 also has lower cost MAE (Table 6), reversing the deployment-study ordering. These selected cases show that earlier agreement need not transfer to new checkpoints and execution conditions; they do not estimate population screening accuracy. The supplement records selection and execution details.

8.3 Transfer across serving contexts

The engine study tests whether the evaluation contract can carry unchanged policies to SGLang and still distinguish feasible from infeasible executions. The six arms are H, H+guard, the deployment study's PPO-1/PPO-2, and two screening checkpoints selected by lowest historical mean cost within Lookup-69's accepted and rejected groups. All 26 runs are technically valid. The all2, guarded, and PPO runs satisfy P1+; H meets every deadline in recovery but leaves one offline job unfinished in each steady repeat.

The historical Lookup-69 proxy falsely rejects eight feasible runs, whereas Lookup-30 and Lookup-SG match all 24 outcomes, including H's two failures (Table 7). Measuring startup locally slightly lowers cost error relative to Lookup-30 but leaves feasibility judgments unchanged. Each model supplies 12 predictions, each compared with two physical repeats.

The arrival study tests this calibration on additional arrivals without refitting. On three new constructed windows, all 24 policy runs are feasible: Lookup-69, Lookup-30, and Lookup-SG agree on 14/24, 22/24, and 24/24, respectively. Lookup-SG thus avoids the false rejections here; this all-feasible cohort does not test detection of infeasible policies.

The 14B study tests remeasured lifecycle timing in Qwen2.5-14B-Instruct/vLLM. H fails all four runs; H+guard passes all four. Replacing 14B-Hist's historical startup with the measured value in 14B-Local raises agreement from 4/8 to 8/8 and lowers cost MAE from 0.4105 to 0.0668 scenario USD. A retrospective 30 s startup contrast, holding all other 14B fields fixed, reaches 6/8: it still falsely accepts H in steady. Each model supplies four policy/window predictions reused across two repeats. Event replay shows how longer startup changes control: 14B-Hist retains queue pressure through cooldown expiry and raises the target to two GPUs, altering later dispatch and deadline outcomes. The supplement traces these feedback paths and distinguishes them from physical-state mismatches that precede restart.

Table 8. Formal held-out results. Each arm has three repeats per window; H has 15 offline deadline misses across its three recovery repeats.

Arm	tech.	offline	online
all2	9/9	9/9 (120/120)	9/9
all1	9/9	9/9 (120/120)	9/9
H	9/9	6/9	9/9
H+guard	9/9	9/9 (120/120)	9/9

These errors also affect deployment choices. Retrospectively requiring predicted P1+ in every window, then minimizing mean predicted cost, selects PPO-2 in the deployment study under Lookup-30; it fails all six physical runs. In the engine study, Lookup-69 selects a feasible policy costing 58.95% more than the lowest-cost feasible policy in the tested pool, which Lookup-30 and Lookup-SG select. Across a 3×3 GPU/API price grid, the deployment study's failing choice persists and the engine study's gap ranges from 16.56% to 80.17%. Supplement Section K reports choices and ties. The interfaces transfer across serving contexts; physical outcomes validate both predictions and their consequences for policy selection.

8.4 Control case study: feasibility and cost

The guard provides a rule-based control case alongside the learned policies. Table 8 summarizes the original held-out study. All2 and all1 satisfy the registered completion target. H is technically valid but fails the P1 gate in all three recovery repeats (116/120, 113/120, and 116/120 offline on time). In that held-out study, the guard repairs this failure without changing the workload or the locked model.

H+guard's window cost is lower than all1 by 21.96%, 48.19%, and 58.08% in steady, recovery, and burst-offline, respectively. Their equal-window mean is 42.74%, above the preregistered 25% P2 line, although steady falls below that stretch target. Including setup and cleanup gives a separate mean saving of 38.86%.

The primary ledger covers the same 2,100 s interval; the deployment ledger also charges setup and cleanup. The supplementary table retains both views, with unchanged prices and event streams.

Before execution, Local's locked recovery prediction identifies four unfinished offline requests; the three physical H repeats leave four, seven, and four unfinished, for 15 total misses. This prediction is interpreted alongside, not

Table 9. Guard activity by recovery repeat. Ticks are window seconds; slack is the minimum pending-offline time to deadline (seconds), before subtracting service time.

Repeat	risk ticks	target raises	tick range	slack range
1	30	1	1982–2011	29–58
2	0	0	–	–
3	29	1	2027–2055	30–58

substituted for, the observed completion evidence.

8.5 Latency–cost trade-off

Resource savings consume online latency margin: per-repeat p99 is about 1.6–2.6 s for always-on references and 15–19 s for both dynamic arms (supplementary tail table). Requests routed to the synthetic API sink wait at least 10 s before admission; the guard protects offline deadlines without removing this delay. All observed p99 values remain below the registered 60 s online deadline, making the saving an explicit SLO-margin-for-cost trade-off.

8.6 Guard activation and mechanism

The guard checks deadline risk using the visible queue, running-job ages, lifecycle delay, and concurrency-four service estimate. Equation 3 retains risk IDs until dispatch or removal from the pending queue and distinguishes risk from a target raise. In the held-out guard study, the logs contain 59 risk-active ticks, all in recovery: the guard raised the target above H twice, with the remaining risk ticks maintaining the already-raised target. Its steady and burst-offline runs have no risk-active ticks. In the 14B study, active guard runs in recovery and steady restart capacity for tail queues below H's scale-out threshold (supplement). Table 9 reports the held-out recovery counts and slack ranges.

8.7 Price sensitivity

Repricing the same H+guard and all1 event streams over 27 window–GPU–API price cells yields one negative saving: steady at half-price GPUs and twice-price API use (-39.59%). Its break-even API multiplier is 1.004. Recovery and burst remain positive throughout the tested grid; the supplement retains the full table. These are scenario sensitivities, not provider billing forecasts.

External checks cover queue-target (B-HPA), Holt-predictive (B-PRED), and queueing-based (B-MMC) baselines, Llama, and Azure arrivals (supplement). Each baseline meets all deadlines in nine runs; H+guard costs less than B-HPA on two known Qwen windows, with higher API use and queue exposure. Retrospective replay of 24 Azure runs gives four archived lookup-based models identical 21/24 agreement, but Lookup-69 falsely accepts three runs while the others reject three feasible runs. H's physical repeats have mixed outcomes in both windows. Error direction therefore matters alongside aggregate agreement.

9 Discussion

FidelityLoop evaluates whether policies meet deadlines and what resources feasible executions consume. Lookup-30 reduces fixed-H resource error, whereas Lookup-69 matches feasibility better in the mixed-policy study and the deployment study. Tests on new checkpoints, SGLang, and a 14B vLLM port show why predictions must be revalidated in each deployment context.

9.1 Validating predictions across levels

Lifecycle validation should follow the controller's clock. For controllers that block scale-down until a target-change cooldown expires, readiness after expiry may permit immediate scale-down even when replay predicts a service interval protected from scale-down. The deployment study exhibits this interaction. Comparing deployment startup timing with controller boundaries is therefore a useful diagnostic; setup and window scale-out should be measured separately. State and action trajectories also need inspection because aggregate resource error can hide temporal misalignment.

The guard shows how a controller can act on deadline risk from observed queues and frozen safety estimates. It repairs H's held-out recovery failure, but H does not read the simulator's startup parameters: the endpoint correction and the guard benefit have distinct causal roles.

9.2 Resource and latency trade-offs

Elastic policies trade online latency margin for lower occupied capacity. The same-runtime comparison with B-HPA quantifies this trade-off in API share, lifecycle-event counts, and queue exposure. Price sensitivity and the Llama and Azure tests identify economic and workload boundaries; meeting one declared SLO does not establish suitability for tighter latency targets or heavier loads.

The supplementary CPU microbenchmark measures `adapter.decide` on reconstructed states, excluding live-serving and full-controller overhead.

Practical validation. Measure lifecycle timing for the deployment, then freeze predictions before physical execution. Check service timing, resource use, and control trajectories separately; report false accepts and false rejects, retaining each physical repeat. Condition savings on meeting deadlines. When arrivals or serving context change, repeat these checks under the same accounting contract.

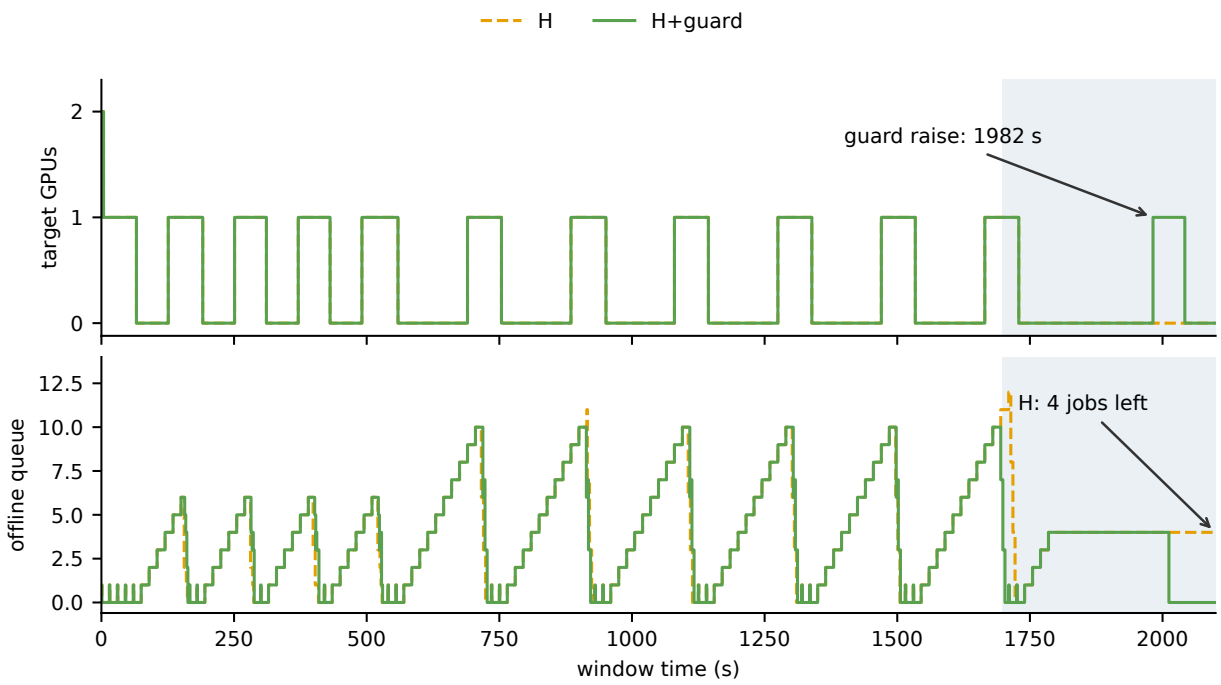


Figure 4. Recovery target and offline queue in repeat 1. The shaded final interval contains the guard raise at 1982 s; H leaves four offline jobs queued, while the guard clears them. Across all three H repeats, 15 jobs miss their deadlines.

10 Scope and Limitations

The synthetic API implements a five-second timer after routing wait, without answer generation or quality evaluation. Scenario costs thus measure the declared resource contract rather than equal-quality commercial substitution. GPU release denotes process and resource release, not physical power-off, energy consumption, or provider billing.

The studies cover two-GPU deployments, Qwen 7B/14B and Llama 8B, vLLM and SGLang, and selected trace-derived windows. Repeats measure execution robustness rather than independent workload coverage. Missing session identities, constructed offline pressure, and nonrandom checkpoint selection limit population-level inference. Screening, engine, and 14B studies reuse known windows; screening and arrival cohorts are all feasible, whereas the engine and 14B cohorts contain two and four infeasible policy runs. The evidence does not establish large-scale heterogeneous deployment or a general ranking of replay models, PPO training environments, or PPO versus the guard; other model families and quantized inference remain untested.

Runtime identities remain separate: deployment drift confounds Legacy-refit–Local, the screening study uses one allocation per window, and the engine study changes the engine and dependencies together. The 14B study isolates startup within the 14B predictor, not the causal effect of model size, engine, or hardware. The same-runtime baseline comparison includes dispatch differences, rather than isolating the guard. Guard repairs provide no general safety guarantee, as the Llama failure illustrates. The supplement retains technical-attempt history and detailed boundaries.

11 Conclusion

FidelityLoop connects replay-based policy selection to physical deadline and resource outcomes through a shared policy interface and aligned execution records. Its measurements and replay contrasts show how startup–cooldown interactions and temporally misplaced capacity can undermine deployment decisions despite accurate service or aggregate-resource estimates.

Across the tested contexts, replay errors can admit failing policies or exclude lower-cost feasible alternatives. Rule-based and learned policies, two serving engines, and an external service model exercise the same validation framework. The released traces and CPU tools let developers reproduce these diagnoses and reuse the validation contract.

REFERENCES

Agrawal, A., Kedia, N., Mohan, J., Panwar, A., Kwatra, N., Gulavani, B. S., Ramjee, R., and Tumanov, A. Vidur: A Large-Scale Simulation Framework for LLM Inference. In *Proceedings of Machine Learning and Systems*, volume 6, pp. 351–366, 2024a. URL https://proceedings.mlsys.org/paper_files/paper/2024/hash/b74a8de47d2b3c928360e0a011f48351-Abstract-Conference.html.

Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B., Tumanov, A., and Ramjee, R. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 117–134, Santa Clara, CA, July 2024b. USENIX Association. ISBN 978-1-939133-40-3. URL https://www.usenix.org/conference/osdi24/presentation/agrawal.

Crankshaw, D., Sela, G.-E., Mo, X., Zumar, C., Stoica, I., Gonzalez, J., and Tumanov, A. InferLine: Latency-Aware Provisioning and Scaling for Prediction Serving Pipelines. In *Proceedings of the 11th ACM Symposium on Cloud Computing*, pp. 477–491, 2020. doi: 10.1145/3419111.3421285. URL https://doi.org/10.1145/3419111.3421285.

Fu, Y., Xue, L., Huang, Y., Brabete, A.-O., Ustiugov, D., Patel, Y., and Mai, L. ServerlessLLM: Low-Latency Serverless Inference for Large Language Models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 135–153, 2024. URL https://www.usenix.org/conference/osdi24/presentation/fu.

Gujarati, A., Karimi, R., Alzayat, S., Hao, W., Kaufmann, A., Vigfusson, Y., and Mace, J. Serving DNNs like Clockwork: Performance Predictability from the Bottom Up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pp. 443–462, 2020. URL https://www.usenix.org/conference/osdi20/presentation/gujarati.

Kabakibo, H. S., Trivedi, A., and Wang, L. Breaking the Ice: Analyzing Cold Start Latency in vLLM. In *Proceedings of Machine Learning and Systems*, volume 8, pp. 1187–1205, 2026. URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/29416b66c2149872b9d1415a3fd2c5e0-Abstract-Conference.html.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626, 2023. doi: 10.1145/3600006.3613165. URL https://doi.org/10.1145/3600006.3613165.

Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, Í., Maleki, S., and Bianchini, R. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pp. 118–132, 2024. doi: 10.1109/isca59077.2024.00019. URL https://doi.org/10.1109/isca59077.2024.00019.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal Policy Optimization Algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

Sun, B., Huang, Z., Zhao, H., Xiao, W., Zhang, X., Li, Y., and Lin, W. Llumnix: Dynamic Scheduling for Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 173–191, 2024. URL https://www.usenix.org/conference/osdi24/presentation/sun-biao.

Wang, Y., Chen, Y., Li, Z., Kang, X., Tang, Z., He, X., Guo, R., Wang, X., Wang, Q., Zhou, A. C., and Chu, X. BurstGPT: A Real-world Workload Dataset to Optimize LLM Serving Systems, 2024. URL https://arxiv.org/abs/2401.17644v3.

Yang, M., Zhang, Z., Wu, M., Yan, J., Sun, H., and Chang, L.-W. Charon: A Unified and Fine-Grained Simulator for Large-Scale LLM Training and Inference. In *Proceedings of Machine Learning and Systems*, volume 8, pp. 94–107, 2026. URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/dbc8ce0fdfcd55172d73fb05dbae07fc-Abstract-Conference.html.

Zhang, C., Yu, M., Wang, W., and Yan, F. MArk: Exploiting Cloud Services for Cost-Effective, SLO-Aware Machine Learning Inference Serving. In *2019 USENIX Annual Technical Conference*, pp. 1049–1062, 2019. URL https://www.usenix.org/conference/atc19/presentation/zhang-chengliang.

Zhang, G., Guo, W., Tan, Z., Guan, Q., and Jiang, H. KIS-S: A GPU-Aware Kubernetes Inference Simulator with RL-Based Auto-Scaling, 2025a. URL https://arxiv.org/abs/2507.07932.

Zhang, G., Vippagunta, S., Nandagopal, R., Raman, S., Xu, J., Pfeiffer, M., Chatterjee, S., Tan, Z., Guo, W., and Jiang, H. AAPA: An Archetype-Aware Predictive Autoscaler

with Uncertainty Quantification for Serverless Workloads on Kubernetes, 2025b. URL https://arxiv.org/abs/2507.05653.

Zhang, H., Tang, Y., Khandelwal, A., and Stoica, I. SHEPHERD: Serving DNNs in the Wild. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation*, pp. 787–808, 2023. URL https://www.usenix.org/conference/nsdi23/presentation/zhang-hong.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. SGLang: Efficient Execution of Structured Language Model Programs. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 62557–62583. Curran Associates, Inc., 2024. doi: 10.52202/079017-2000. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/724be4472168f31ba1c9ac630f15dec8-Paper-Conference.pdf.